

Algoritmos De Búsqueda y Algoritmos De Ordenamiento Básico

Oscar Tabares y Diego Escobar

Centro de Formación Integral para el Trabajo (CEFIT)

Técnica laboral en auxiliar análisis y programación de software

Jorge Eliecer Giraldo Lipeda

Envigado, Colombia

23 de noviembre de 2025

Resumen

Este trabajo realiza una comparación sistemática de los algoritmos de búsqueda y ordenamiento de mayor uso en el ámbito de la programación. Se analizan en profundidad los métodos de búsqueda lineal y binaria, junto con los algoritmos de ordenamiento Insertion Sort, Merge Sort y Quick Sort. Para cada uno de ellos, se describe su funcionamiento, se especifica su complejidad computacional en el mejor, promedio y peor de los casos, y se discuten sus escenarios de aplicación óptimos y sus principales limitaciones. Como complemento práctico, se incluye la implementación de ejemplos codificados en Python que ilustran paso a paso el proceso de ejecución de cada algoritmo. El objetivo del estudio es proporcionar una base fundamentada que facilite la selección del algoritmo más eficiente y adecuado para una necesidad de programación específica.

Palabras clave: algoritmos, búsqueda, ordenamiento, complejidad computacional, Python, eficiencia.

La Importancia de la Selección de Algoritmos en el Procesamiento

Eficiente de Datos

En el mundo de la computación, los algoritmos de búsqueda y ordenamiento son clave para trabajar con datos de forma rápida y ordenada. Elegir el algoritmo correcto puede hacer que una aplicación responda en milisegundos, mientras que uno inadecuado puede hacer que tarde minutos o incluso horas.

En este trabajo nos enfocamos en dos tipos de algoritmos: los de búsqueda, que nos ayudan a encontrar elementos dentro de un conjunto de datos, y los de ordenamiento, que se encargan de organizarlos según algún criterio. Vamos desde los más sencillos hasta algunos más avanzados, evaluando su rendimiento con la notación Big O y mostrando ejemplos de cómo se programan.

El objetivo es que, al terminar de leer, tanto desarrolladores como estudiantes sepan elegir el algoritmo ideal para cada situación, teniendo en cuenta el tamaño de los datos, si cambian mucho o poco, y qué tan rápido deben funcionar.

Algoritmos De Búsqueda

Búsqueda Lineal

¿Qué es y para qué sirve?

Es el método más simple para buscar un valor en una lista: se recorre elemento por elemento hasta encontrarlo. Es fácil de entender y no pide que los datos estén ordenados, así que es útil cuando trabajamos con conjuntos pequeños o que no tienen un orden específico.

Aplicaciones principales:

- Buscar elementos en listas pequeñas
- Verificar existencia de datos en colecciones
- Procesar datos no ordenados
- Implementaciones simples donde la eficiencia no es crítica

¿Cómo funciona paso a paso?

El algoritmo de búsqueda lineal sigue un proceso sencillo:

- Comienza desde el primer elemento de la lista
- Compara el elemento actual con el valor que se busca
- Si el elemento coincide con el objetivo, devuelve su posición (o índice)
- Si el elemento no coincide, pase al siguiente elemento
- Repite los pasos 2 a 4 hasta encontrar el objetivo o hasta que finalice la lista
- Si se llega al final de la lista sin encontrar el objetivo, devuelve -1 o indica que el elemento no está presente

Ventajas:

- Fácil de implementar y entender
- Funciona con cualquier tipo de datos
- No necesita datos ordenados
- Mantiene el orden original de los datos
- Comportamiento lineal fácil de analizar

Desventajas:

- Ineficiencia: $O(n)$ en tiempo - lento para listas grandes
- Escalabilidad pobre: Tiempo crece proporcionalmente con el tamaño
- No optimizable: No aprovecha datos ordenados
- Recurso intensivo: Muchas comparaciones en el peor caso

Un ejemplo funcional en Python explicado paso a paso.

Puedes ver un código funcional y explicado paso a paso en:

<https://github.com/diegonba224-creator/Algoritmos/blob/main/3.1%20B%C3%BAsqueda%20Lineal>

Búsqueda binaria

Concepto y condiciones necesarias (lista ordenada)

La búsqueda binaria es un potente algoritmo diseñado para encontrar de forma eficiente un valor en un conjunto de datos ordenado. La idea central de la búsqueda binaria es sencilla: En lugar de comprobar cada elemento del conjunto de datos uno a uno, como en una búsqueda lineal, la búsqueda binaria reduce el intervalo de búsqueda a la mitad con cada paso, lo que hace que el proceso sea mucho más rápido.

Para que la búsqueda binaria funcione, el conjunto de datos debe estar ordenado de forma ascendente o descendente. Esto es obligatorio porque el algoritmo se basa en el orden de los elementos para determinar en qué mitad del conjunto de datos debe buscar a continuación. Si los datos no están ordenados, la búsqueda binaria no puede localizar el valor objetivo.

¿Cómo funciona la técnica de dividir y conquistar?

Es una técnica algorítmica que resuelve un problema dividiéndolo en subproblemas más pequeños, resolviendo cada subproblema recursivamente, y combinando las soluciones.

Descripción paso a paso del proceso.

Imagina que estás jugando a un juego de adivinanzas en el que tienes que encontrar un número concreto entre 1 y 100. Puedes intentar adivinar al azar, pero, en el peor de los casos, puede que necesites 100 intentos para llegar a la respuesta correcta.

Un método más breve podría ser elegir el número del medio, 50, y preguntar si el número objetivo es mayor o menor. Si tu número objetivo es mayor, puedes ignorar todo lo que esté por

debajo de 50 y repetir esta tarea con los números 51-100. Puedes continuar hasta que encuentres el número correcto.

Al reducir a la mitad las posibilidades cada vez, te centras rápidamente en el objetivo. Con este método, incluso en el peor de los casos, solo harían falta como máximo 7 intentos para encontrar el número correcto. Esta estrategia es la esencia de la búsqueda binaria.

Ventajas:

- Alta eficiencia: $O(\log n)$ vs $O(n)$ de búsqueda lineal
- Óptimo para listas grandes
- Reduce rápidamente el espacio de búsqueda
- Predecible y estable

Desventajas:

- Requiere lista ordenada
- Si la lista cambia mucho, hay que reordenarla seguido
- Más complejo de programar que la búsqueda lineal

Un ejemplo funcional en Python explicado paso a paso.

Código completo y explicado aquí:

<https://github.com/diegonba224-creator/Algoritmos/blob/main/3.2%20B%C3%BAsqueda%20Binaria>

Algoritmos de Ordenamiento Básico

Insertion Sort (Ordenamiento por Inserción)

¿Qué problema resuelve?

El algoritmo de ordenamiento por inserción es un algoritmo simple pero eficiente. Funciona dividiendo la lista en dos partes, una parte ordenada y otra desordenada, a medida que se recorre la lista desordenada, se insertan elementos en la posición correcta en la parte ordenada. Resuelve el problema de ordenar elementos en orden ascendente o descendente.

Este algoritmo ordena una lista como lo harías con una mano de cartas: vas insertando cada carta en su lugar correcto dentro de la parte ya ordenada.

Es particularmente útil cuando:

- El conjunto de datos es pequeño.
- Los datos están casi ordenados.
- Se necesita un algoritmo estable (mantiene el orden relativo de elementos iguales).

Explicación del funcionamiento.

El algoritmo funciona dividiendo la lista en dos partes:

1. Parte ordenada (inicialmente solo el primer elemento)
2. Parte desordenada (el resto de los elementos)

Pasos del algoritmo:

1. Comenzar con el segundo elemento (índice 1)
2. Comparar con los elementos de la parte ordenada
3. Insertar en la posición correcta
4. Repetir hasta que todos los elementos estén ordenados

Ventajas:

- **Baja sobrecarga:** Requiere menos comparaciones y movimientos que algoritmos como el ordenamiento de burbuja, lo que lo hace más eficiente en términos de intercambios de elementos.
- **Simplicidad:** el ordenamiento por inserción es uno de los algoritmos de ordenamiento más simples de implementar y entender. Esto lo hace adecuado para enseñar conceptos básicos de ordenamiento.

Desventajas:

- **Ineficiencia en listas grandes:** A medida que el tamaño de la lista aumenta, el rendimiento del ordenamiento por inserción disminuye. Su complejidad cuadrática de $O(n^2)$ lo hace ineficiente para las listas grandes.
- **No escalable:** Al igual que otros algoritmos de complejidad cuadrática, el ordenamiento por inserción no es escalable para listas grandes, ya que su tiempo de ejecución aumenta considerablemente con el tamaño de la lista.

Un ejemplo funcional en Python explicado paso a paso.

Implementación paso a paso:

<https://github.com/diegonba224-creator/Algoritmos/blob/main/3.3%20Insertion%20Sort>

Merge Sort

¿Qué es y por qué se considera eficiente?

El Merge Sort, u "ordenamiento por mezcla", es como un experto en organizar información que siempre trabaja con la misma eficiencia, sin importar qué tan desordenados estén los datos desde el principio.

A diferencia de otros métodos como el Insertion Sort o Bubble Sort - que pueden volverse muy lentos cuando tienen que ordenar grandes cantidades de información desordenada -, el Merge Sort mantiene su alto rendimiento en cualquier situación.

Explicación del algoritmo “divide y vencerás”

Se parte la lista en dos mitades una y otra vez, hasta que solo queden listas de un solo elemento. Una lista con un elemento ya está ordenada por definición.

Una vez dividido todo en sus partes más pequeñas, cada una de esas minilistas (que ya están ordenadas) se convierte en la base para reconstruir la lista final.

¿Cómo se realiza la etapa de mezcla?

La etapa de mezcla (merge) es el corazón del algoritmo:

1. **Comparar progresivamente:** Se comparan los primeros elementos de dos sublistas ordenadas.
2. **Seleccionar el menor:** Se toma el elemento más pequeño y se coloca en la lista resultante.
3. **Avanzar índices:** Se avanza en la sublista de donde se tomó el elemento.
4. **Repetir:** Hasta que una sublista se vacíe.
5. **Copiar restantes:** Los elementos restantes se copian directamente.

Ventajas y limitaciones.

Ventajas

- **Rendimiento Consistente $O(n \log n)$:** Siempre mantiene esta eficiencia, sin importar si los datos están totalmente desordenados, parcialmente ordenados o invertidos.
- **Algoritmo Estable:** Mantiene el orden relativo de elementos con claves iguales, lo que es crucial cuando el orden original de elementos equivalentes es significativo.
- **Predecible:** El rendimiento es predecible y no depende de factores aleatorios o del orden de entrada, facilitando el análisis de rendimiento.

Limitaciones

- **Alto Consumo de Memoria $O(n)$:** Cada fusión requiere crear listas temporales
- **No es In-Place:** No ordena en el mismo espacio de memoria, sino que requiere memoria adicional
- **Implementación Más Compleja:** Comparado con algoritmos simples como Bubble Sort o Insertion Sort, la implementación es más compleja y requiere manejar cuidadosamente la etapa de mezcla.
- **No Adaptativo:** No se beneficia de listas parcialmente ordenadas; siempre realiza el mismo número de operaciones independientemente del orden inicial.

Un ejemplo funcional en Python explicado paso a paso

Código y explicación detallada:

<https://github.com/diegonba224-creator/Algoritmos/blob/main/3.4%20Merge%20Sort>

Quick Sort

Explicación general.

Quick Sort también usa “dividir y conquistar”, pero de otra forma: elige un pivote y reorganiza la lista para que los elementos menores queden a la izquierda y los mayores a la derecha. Luego repite el proceso con cada sublista. Es muy rápido en la práctica.

Características principales:

- Algoritmo de ordenamiento por comparación
- Paradigma "divide y vencerás"
- Generalmente más rápido que Merge Sort y Heap Sort
- Ordenamiento in-place (no requiere memoria adicional significativa)

¿Qué es un pivote y cómo se escoge?

El pivote es un elemento de la lista alrededor del cual se reorganizan los demás elementos. Los elementos menores van a su izquierda y los mayores a su derecha. Esta elección del pivote es crucial para el rendimiento del algoritmo.

El **pivote** es el elemento de referencia alrededor del cual se organiza la lista. Imagínalo como el "punto de equilibrio" que divide la lista en tres partes:

- **Izquierda:** Elementos MENORES que el pivote
- **Pivote:** En su posición final correcta
- **Derecha:** Elementos MAYORES que el pivote

Estrategias para Elegir el Pivote

1. Primer o Último Elemento (Ingenua): Estrategia más simple pero problemática. Si la lista ya está ordenada o invertida, el rendimiento se desploma a $O(n^2)$.

2. Elemento del Medio: Funciona bien con listas ya ordenadas.

3. Selección Aleatoria

4. Mediana de Tres (La más efectiva): Devuelve la mediana de los tres. Se comparan el primer, último y elemento central, y se elige el valor del medio.

Funcionamiento del particionamiento.

El particionamiento es la operación clave de Quick Sort que:

1. Selecciona un pivote
2. Reorganiza la lista colocando elementos menores a la izquierda y mayores a la derecha
3. Coloca el pivote en su posición final correcta

Ventajas

- Eficiencia en tiempo promedio.
- Uso de espacio óptimo.
- Implementación intuitiva.

Un ejemplo funcional en Python explicado paso a paso.

Implementación completa:

<https://github.com/diegonba224-creator/Algoritmos/blob/main/3.5%20Quick%20Sort>

Explicación de Complejidades Computacionales

Algoritmo	Mejor	Promedio	Peor	Espacio	Estable
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Sí
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗ No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Sí

$O(n)$ - Complejidad Lineal: si la lista tiene 1000 elementos, el algoritmo hace aproximadamente 1000 operaciones y el tiempo crece proporcionalmente al tamaño de entrada.

$O(n^2)$ - Complejidad Cuadrática: si la lista tiene 1000 elementos, el algoritmo hace aproximadamente 1,000,000 operaciones, el tiempo crece con el cuadrado del tamaño de entrada.

$O(n \log n)$ - Complejidad Lineal-Logarítmica: si la lista tiene 1000 elementos, el algoritmo hace aproximadamente 10,000 operaciones, lo cual es muy eficiente para listas grandes.

$O(\log n)$ - Complejidad Logarítmica: si la lista tiene 1000 elementos, el algoritmo hace aproximadamente 10 operaciones. Es muy eficiente pero el tiempo crece muy lentamente.

$O(1)$ - Complejidad Constante: siempre toma el mismo tiempo, sin importar el tamaño de la entrada.

Hallazgos y conclusiones

Principales hallazgos

- No hay un algoritmo perfecto para todo: La elección depende del tamaño de los datos, la frecuencia de cambios y si necesitas estabilidad.
- Siempre hay trade-offs: Velocidad vs. memoria, simplicidad vs. eficiencia.
- A veces conviene ordenar primero: Invertir tiempo en ordenar (para luego usar búsqueda binaria) puede ahorrar mucho después.
- Quick Sort suele ser más rápido en la práctica: Aunque en el peor caso es lento, en promedio es excelente.

Recomendaciones Prácticas

- Conjuntos pequeños (<50 elementos): Insertion Sort por simplicidad.
- Conjuntos medianos (50-1000 elementos): Quick Sort por eficiencia práctica.
- Conjuntos grandes (>1000 elementos): Merge Sort por garantía de rendimiento.
- Datos frecuentemente actualizados: Búsqueda lineal evita costo de reordenamiento.
- Datos estáticos: Búsqueda binaria maximiza eficiencia de consulta.
- Requisitos de estabilidad: Merge Sort o Insertion Sort según tamaño.

Referencias

DataCamp. (2023). *Búsqueda lineal en Python*

<https://www.datacamp.com/es/tutorial/linear-search-python>

DataCamp. (2023). *Búsqueda binaria en Python*

<https://www.datacamp.com/es/tutorial/binary-search-python>

4Geeks. (2023). *Algoritmos de ordenamiento y búsqueda en Python*

<https://4geeks.com/es/lesson/algoritmos-de-ordenamiento-y-busqueda-en-python>

DataCamp. (2023). *Python Merge Sort Tutorial*

<https://www.datacamp.com/es/tutorial/python-merge-sort-tutorial>

Prezi. (2023). *Ordenamiento rápido (Quick Sort) en Python*

<https://prezi.com/p/kmakqdilgq3n/ordenamiento-rapido-quick-sort-en-python/>

Apéndice

Repositorio de Implementaciones

Todas las implementaciones referenciadas en este trabajo están disponibles en el repositorio

GitHub:

<https://github.com/diegonba224-creator/Algoritmos/tree/main>